

IPO Base Scanner

Complete Recreation Guide

Application	IPO Base Scanner - High-Probability Breakout Identification
Purpose	Scan Indian IPO stocks using a 6-pillar scoring methodology to identify base breakouts
Tech Stack	Next.js 16, TypeScript, Tailwind CSS 4, shadcn/ui, Prisma SQLite, Zustand, Recharts
Data Source	Yahoo Finance v8 Chart API (real-time + historical)
Database	SQLite via Prisma ORM (3 models: IpoStock, DailyPrice, WeeklyPrice)
Deployment	Standalone build (output: "standalone"), Caddy reverse proxy on port 81
Version	1.0 - May 2026

This document contains every detail needed to recreate the IPO Base Scanner application from scratch, including complete architecture, all API endpoints, scoring algorithms, database schema, UI component specifications, configuration files, and the full list of seed stocks with their data.

Table of Contents

1. Application Overview & Methodology
2. Architecture & Data Flow
3. Technology Stack & Dependencies
4. Project Structure & File Map
5. Database Schema (Prisma)
6. API Routes - Complete Specification
7. 6-Pillar Scoring Algorithm
8. Seed Data - 20 Indian IPO Stocks
9. State Management (Zustand Store)
10. UI Components Specification
11. Configuration Files
12. Yahoo Finance Integration
13. Deployment Configuration
14. Known Issues & Planned Features

1. Application Overview & Methodology

The IPO Base Scanner is a web-based application designed to identify high-probability IPO base breakouts among Indian stocks listed on the NSE (National Stock Exchange) and BSE (Bombay Stock Exchange). It implements the "IPO Growth Playbook" methodology, which analyzes newly listed stocks through a structured 6-pillar scoring system to determine whether an IPO stock is forming a favorable base pattern that could lead to a significant price breakout. The application provides real-time market data integration via Yahoo Finance, allowing users to refresh prices and historical data on demand, and then run the scoring algorithm to update all stock evaluations with the latest information.

The core philosophy behind the scanner is that IPO stocks often follow predictable patterns after listing: an initial excitement phase drives prices up, followed by a decline as early investors exit, then a base formation period where the stock consolidates with declining volume and tightening price ranges (the Volatility Contraction Pattern or VCP). When the stock eventually breaks out of this base on strong volume, it presents a high-probability trading opportunity. The 6-pillar scoring system quantifies each aspect of this pattern to produce a composite score and a recommendation (Strong Buy, Buy, Watch, or Avoid) for each stock in the watchlist.

Key Features

- 6-pillar scoring algorithm with 0-100 scores per pillar and composite total score
- Real-time Yahoo Finance integration for prices and 6-month historical OHLCV data
- Stock watchlist management: add via Yahoo search, manual entry, or delete existing
- Filtering by base status (Forming/Breakout/Advancing/Failed), recommendation, and minimum score
- Interactive price charts with daily/weekly toggle, pivot/support/IPO reference lines
- IPO Base Checklist showing pass/fail for each criterion with descriptive explanations
- Risk assessment with Low/Moderate/Elevated/High ratings and detailed commentary
- Responsive dark theme UI built with shadcn/ui components and Framer Motion animations
- SQLite database with Prisma ORM for persistent storage of stocks, daily prices, and weekly prices
- Auto-seeding on first load with 20 pre-configured Indian IPO stocks across 3 pattern categories

2. Architecture & Data Flow

The application follows a standard Next.js App Router architecture with client-side and server-side responsibilities clearly separated. The frontend is a single-page application rendered as a React client component, using Zustand for local UI state (filters, selections, loading flags) and React Query (TanStack) for server data fetching and caching. All data persistence happens server-side through API routes that interact with a SQLite database via Prisma ORM.

Data Flow Diagram

Layer	Technology	Responsibility
UI Components	React + shadcn/ui + Framer Motion	Rendering stock cards, charts, filters, dialogs
State (UI)	Zustand Store	Filters, selected stock, loading flags
Data Fetching	TanStack React Query	Cache, auto-refetch, loading/error states
API Routes	Next.js Route Handlers	CRUD operations, scanning, Yahoo Finance proxy
Database	SQLite + Prisma ORM	Persistent stock/price storage
External API	Yahoo Finance v8 Chart API	Real-time prices and historical OHLCV data

Page Load Flow

1. On first load, the app checks if the database has stocks by calling GET /api/ipo-stocks. 2. If empty, it calls POST /api/seed to populate with 20 default stocks and synthetic price data. 3. React Query fetches stocks with current filters applied via GET /api/ipo-stocks?status=...&recommendation;=... 4. User clicks "Run Scanner" which calls POST /api/scan to score all stocks using the 6-pillar algorithm. 5. User clicks "Refresh Prices" to fetch latest prices from Yahoo Finance via POST /api/market-data. 6. After refresh, the scanner auto-runs to update scores with the new price data. 7. Clicking a stock card opens a detail dialog that fetches full data via GET /api/ipo-stocks/[id].

3. Technology Stack & Dependencies

Package	Version	Purpose
next	^16.1.1	React framework with App Router, SSR, API routes
react	^19.0.0	UI library
@prisma/client	^6.11.1	ORM for SQLite database
prisma	^6.11.1	Schema management, migrations, client generation
zustand	^5.0.6	Lightweight state management for UI state
@tanstack/react-query	^5.82.0	Server state management, caching, auto-refetch
recharts	^2.15.4	Charting library for price charts (ComposedChart)
framer-motion	^12.23.2	Animation library for card transitions, entrance effects
lucide-react	^0.525.0	Icon library (Activity, TrendingUp, etc.)

tailwindcss	^4	Utility-first CSS framework
sharp	^0.34.3	Image optimization for Next.js
sonner	^2.0.6	Toast notifications

shadcn/ui Components Used

The application uses the following shadcn/ui components (New York style, neutral base, CSS variables): Button, Badge, Dialog, Input, Select, Slider, Separator, ScrollArea, Tabs, Tooltip, Toaster, Card, and Progress. The shadcn/ui configuration is set with style="new-york", RSC enabled, TypeScript, and Lucide icons. These components are located in src/components/ui/ and were installed via the shadcn CLI.

4. Project Structure & File Map

src/

app/

`globals.css` -- Tailwind CSS 4 imports + light/dark theme CSS variables (oklch color space)

`layout.tsx` -- Root layout: Geist fonts, Toaster, metadata, dark class on html

`page.tsx` -- Main page: IpoScannerApp client component with seed/scan/refresh logic

api/

`route.ts` -- GET /api: Health check endpoint

`seed/route.ts` -- POST: Seed 20 stocks with synthetic OHLCV; DELETE: Clear all data

`scan/route.ts` -- POST: Run 6-pillar scoring algorithm on all stocks

`market-data/route.ts` -- POST: Yahoo Finance integration (search/prices/full/single modes); GET: Status

ipo-stocks/

`route.ts` -- GET: List+filter stocks+stats; POST: Add stock; DELETE: Remove stock

`[id]/route.ts` -- GET: Single stock detail with daily/weekly prices

lib/

`db.ts` -- Prisma client singleton (stored on globalThis)

`ipo-store.ts` -- Zustand store: filters, selectedStock, scanning/loading flags

`utils.ts` -- `cn()` utility (clsx + twMerge)

components/

ipo-scanner/

`scanner-provider.tsx` -- React Query provider (staleTime: 30s, retry: 1)

`header.tsx` -- App header with title, stats row, action buttons

`filter-bar.tsx` -- Search, status/rec/sort selects, min score slider

`stock-grid.tsx` -- Responsive grid with skeleton loading and empty state

`stock-card.tsx` -- Individual stock card with circular score, mini bars, badges

`score-indicator.tsx` -- ScoreIndicator (circular SVG) and MiniScoreBar components

`stock-detail.tsx` -- Full detail dialog: info grid, scores, chart, checklist, risk

`price-chart.tsx` -- Recharts ComposedChart: Area (price) + Bar (volume) + reference lines

`checklist.tsx` -- 6-item pass/fail checklist with threshold-based evaluation

`stock-list-manager.tsx` -- Add/remove stocks dialog with Yahoo search and manual entry

`ui/` -- 45+ shadcn/ui component files

hooks/

`use-mobile.ts` -- `useIsMobile()` hook (768px breakpoint)

`use-toast.ts` -- Toast notification hook

prisma/

`schema.prisma` -- Database schema (IpoStock, DailyPrice, WeeklyPrice)

`db/custom.db` -- SQLite database file

`.env` -- DATABASE_URL=file:/home/z/my-project/db/custom.db

`package.json` -- Dependencies and scripts

`next.config.ts` -- output: "standalone", ignoreBuildErrors, reactStrictMode: false

`tailwind.config.ts` -- darkMode: "class", CSS variable-based theme

`tsconfig.json` -- ES2017 target, bundler resolution, @/* path alias

`components.json` -- shadcn/ui config

`Caddyfile` -- Reverse proxy port 81 to localhost:3000

5. Database Schema (Prisma)

The application uses SQLite as its database, managed through Prisma ORM. The schema defines three models: `IpoStock` (the main stock entity with all metadata and scores), `DailyPrice` (OHLCV price bars at daily resolution), and `WeeklyPrice` (OHLCV price bars at weekly resolution, aggregated from daily). Prices are linked to stocks via a foreign key with cascade delete, ensuring that removing a stock also removes all its associated price data.

IpoStock Model

Field	Type	Default	Description
id	String @id	cuid()	Unique identifier
symbol	String @unique	-	Stock ticker (e.g., MTAR, ZOMATO)
name	String	-	Full company name
sector	String	-	Industry sector
ipoDate	String	-	IPO listing date
ipoPrice	Float	-	IPO issue price
ipoOpenPrice	Float	-	First day opening price
ipoDayLow	Float	-	First day low
ipoDayHigh	Float	-	First day high
currentPrice	Float	-	Latest traded price
listingGainPct	Float	-	Listing day gain percentage
marketCap	String	-	Market cap (e.g., "2,750 Cr")
exchange	String	NSE	Exchange: NSE or BSE
horizontalPivotScore	Int	0	Score 0-100
volumeDryUpScore	Int	0	Score 0-100
breakoutVolumeScore	Int	0	Score 0-100
vcpScore	Int	0	Score 0-100
priceVsIpoLowScore	Int	0	Score 0-100
weeklyConvictionScore	Int	0	Score 0-100
totalScore	Int	0	Average of 6 pillars
baseStatus	String	FORMING	FORMING/BREAKOUT/ADVANCING/FAILED
baseWeeks	Int	0	Number of weeks in base
pivotLevel	Float?	null	Resistance/pivot price level
supportLevel	Float?	null	Support price level
breakoutDate	String?	null	Date of breakout
recommendation	String	WATCH	STRONG_BUY/BUY/WATCH/AVOID
dailyPrices	DailyPrice[]	-	Relation: daily OHLCV bars
weeklyPrices	WeeklyPrice[]	-	Relation: weekly OHLCV bars

DailyPrice & WeeklyPrice Models

Both models share the same structure: `id` (String @id, cuid()), `stockId` (String, foreign key to `IpoStock` with `cascade delete`), `date` (String, e.g., "2024-03-04"), `open` (Float), `high` (Float), `low` (Float), `close` (Float), `volume` (Float). Both have a composite index on `[stockId, date]` for efficient querying. The stock relation uses `@relation(fields: [stockId], references: [id], onDelete: Cascade)`.

6. API Routes - Complete Specification

Method	Path	Purpose	Key Parameters
GET	/api	Health check	None
POST	/api/seed	Seed 20 stocks with synthetic price data	None (uses hardcoded STOCKS array)
DELETE	/api/seed	Clear all stocks and prices from DB	None
GET	/api/ipo-stocks	List stocks with filters and stats summary	status, recommendation, minScore, sortBy, search (query params)
POST	/api/ipo-stocks	Add new stock with synthetic price data	symbol, name, sector, ipoDate, ipoPrice (required body fields)
DELETE	/api/ipo-stocks	Remove stock and its price data	id or symbol (body)
GET	/api/ipo-stocks/[id]	Single stock detail with all daily/weekly prices	id (path param, Next.js 16 Promise pattern)
POST	/api/scan	Run 6-pillar scoring on all stocks	None (scores all stocks in DB)
POST	/api/market-data	Yahoo Finance integration	mode: search prices full single, query, stockId, fullHistory (body)
GET	/api/market-data	Market data status for all stocks	None

Market Data API Modes (POST /api/market-data)

Mode	Behavior
search	Searches Yahoo Finance for Indian stocks (.NS/.BO). Falls back to direct symbol validation via chart API. Returns: symbol, yahooSymbol, name, exchange, sector.
prices	Refreshes latest price for ALL stocks. Uses Yahoo 5d chart API. Also updates last daily price bar so scanner does not override with stale data. 200ms delay between stocks to avoid rate limiting.
full	Replaces ALL price data with 6-month historical from Yahoo Finance. Deletes old daily/weekly prices, creates new ones. Aggregates daily to weekly. 300ms delay between stocks.
single	Refreshes one stock by stockId. With fullHistory=true: replaces all price data with 6-month history. Without: just fetches latest price. Accepts customSymbol for Yahoo Finance symbol override.

7. 6-Pillar Scoring Algorithm

The scoring engine is implemented in POST /api/scan (src/app/api/scan/route.ts). It iterates over all stocks in the database, computes 6 individual pillar scores (each 0-100), derives a total score as the average of all pillars, then determines the base status and recommendation. Each pillar uses a point-based system where specific conditions earn points, and the total is capped at 100. The algorithms analyze both daily and weekly OHLCV data.

Pillar	Max Points	What It Measures	Key Conditions
1. Horizontal Pivot	100	Flat resistance level detection	+40: 8+ weeks near resistance (95% of max high); +30: Touched 3+ times at 98% of max; +30: Price within 5% of pivot
2. Volume Dry-Up	100	Declining volume indicating reduced supply	+30: Volume declining 3 consecutive weeks; +40: Last 2 weeks below 60% of average; +30: Recent minimum volume bar visible
3. Breakout Volume	100	Volume surge on breakout confirming demand	+40: Latest day volume 3x average; +30: 2+ days breaching prior high with 1.5x volume; +30: Up-day volume 1.5x down-day volume
4. VCP Pattern	100	Volatility Contraction Pattern (narrowing ranges)	+40: Range tightness ratio $\geq 3\times$ (first chunk vs last); +30: 2+ contractions in successive chunks; +30: Current range $< 5\%$ of price
5. Price vs IPO Low	100	Position relative to IPO reference levels	+40: Price above IPO open $\times 1.1$; +30: Price above IPO day low $\times 1.2$; +30: Higher lows forming in recent 30 days
6. Weekly Conviction	100	Weekly closes above pivot with volume confirmation	+40: 3+ of last 4 weekly closes above pivot; +30: 3+ weekly closes above key level (92% of pivot); +30: Up-week volume 1.5x down-week volume

Derived Values

Total Score = Average of all 6 pillar scores (rounded to integer). This is the primary metric displayed on stock cards and used for filtering/sorting.

Base Status is determined as follows: BREAKOUT if totalScore ≥ 60 AND breakoutVolume ≥ 45 ; ADVANCING if totalScore ≥ 50 AND weeklyConviction ≥ 35 ; FAILED if totalScore < 30 ; FORMING otherwise (default for stocks still consolidating).

Recommendation is determined as: STRONG_BUY if totalScore ≥ 65 ; BUY if totalScore ≥ 55 AND baseStatus is BREAKOUT or ADVANCING; AVOID if totalScore < 30 ; WATCH otherwise (default). Additional computed fields: baseWeeks = number of weekly price bars (capped at 24); pivotLevel = max of all weekly highs; supportLevel = min of last 8 weekly lows.

8. Seed Data - 20 Indian IPO Stocks

The seed route creates 20 Indian IPO stocks divided into three categories: 10 Success Stories (pattern="success") that simulate stocks with completed base breakouts, 5 Warning Signs (pattern="failed") that represent stocks with broken base patterns, and 5 Forming/Neutral (pattern="forming") that represent stocks currently building bases. Each stock comes with synthetic OHLCV price data generated using a phase-based random walk that models the typical IPO lifecycle. The price generator uses configurable phases with different trend, volatility, and volume parameters for each stage of the pattern.

Symbol	Name	Sector	IPO Price	Current	Score	Status	Rec
MTAR	MTAR Technologies	Defense	574	895	90	BREAKOUT	STRONG_BUY
HARIOMPIPE	Hariom Pipe Industries	Steel	153	310	86	BREAKOUT	STRONG_BUY
CRAFTSMAN	Craftsman Automation	Engineering	1490	2350	81	ADVANCING	BUY
LIKHITHA	Likhitha Infrastructure	Infra	120	245	82	ADVANCING	BUY
ETHOS	Ethos Ltd	Luxury Retail	878	1380	80	BREAKOUT	BUY
METROBRAND	Metro Brands Ltd	Footwear	490	720	75	ADVANCING	BUY
VENUSPIPES	Venus Pipes & Tubes	Steel	326	520	78	BREAKOUT	BUY
HAPPSTMINDS	Happiest Minds Tech	IT Services	166	310	81	ADVANCING	BUY
STOVEKRAFT	Stove Kraft Ltd	Consumer	385	580	75	ADVANCING	BUY
RAINBOW	Rainbow Childrens Medicare	Healthcare	542	780	74	BREAKOUT	BUY
ZOMATO	Zomato Ltd	Internet	76	58	17	FAILED	AVOID
NAZARA	Nazara Technologies	Gaming	1101	680	19	FAILED	AVOID
NURECA	Nureca Ltd	Healthcare	400	290	10	FAILED	AVOID
ADANIWILMAR	Adani Wilmar Ltd	FMCG	230	310	26	FAILED	AVOID
CARTRADE	CarTrade Tech Ltd	Auto Classifieds	1618	980	15	FAILED	AVOID
LATENTVIEW	Latent View Analytics	Analytics	197	340	52	FORMING	WATCH
PARAS	Paras Defence & Space	Defense	175	380	41	FORMING	WATCH
NYKAA	FSN Ecommerce (Nykaa)	Beauty	1125	175	40	FORMING	WATCH
PHARMEASY	API Holdings (PharmEasy)	E-pharmacy	560	380	27	FORMING	WATCH
DELHIVERY	Delhivery Ltd	Logistics	487	420	44	FORMING	WATCH

Price Generation Algorithm

The seed route generates synthetic OHLCV data using a phase-based random walk. For "success" pattern stocks, the phases are: IPO excitement (5% of days, +1% trend, 4% vol), decline (20%, -0.8% trend), wide base (25%, +0.1% trend), tightening base (25%, +0.2% trend, declining volume), VCP tight phase (15%, +0.1% trend, very low volume), and breakout (10%, +1.5% trend, high volume surge). "Failed" pattern uses decline, failed rally, distribution, and breakdown phases. "Forming" pattern uses decline and early consolidation. Weekends are skipped. The last 10 prices are gently adjusted to match the target currentPrice. Weekly prices are aggregated from daily by grouping into Monday-based weeks.

9. State Management (Zustand Store)

The application uses Zustand for client-side UI state management. The store (`src/lib/ipo-store.ts`) manages filter state, the currently selected stock, and various loading flags. Server data (stocks, prices) is managed by TanStack React Query, not Zustand. The store interface is defined as `IpoStore` with the following fields:

Field	Type	Default	Purpose
<code>filters.status</code>	string	ALL	Filter by base status
<code>filters.recommendation</code>	string	ALL	Filter by recommendation
<code>filters.minScore</code>	number	0	Minimum total score filter
<code>filters.sortBy</code>	string	<code>totalScore</code>	Sort field (6 options)
<code>filters.search</code>	string	"	Search by symbol/name
<code>selectedStockId</code>	string null	null	Currently selected stock for detail view
<code>isScanning</code>	boolean	false	Scanner running flag
<code>isSeeded</code>	boolean	false	Database seeded flag
<code>isRefreshingData</code>	boolean	false	Market data refresh flag
<code>isManageStocksOpen</code>	boolean	false	Stock manager dialog open

10. UI Components Specification

The UI consists of 11 custom components in `src/components/ipo-scanner/`, all built as React client components using "use client" directive. The design follows a dark theme (slate-950 background, emerald accents) with glass-morphism effects, gradient overlays, and Framer Motion animations.

Component	Key Features
Header	Gradient bg with blur decorations. Stats grid (Total/Strong Buy/Watch/Avoid). 4 action buttons: Manage Stocks, Refresh Prices, Full Refresh, Run Scanner. Framer Motion entrance animation.
FilterBar	Search input with icon. Status select (5 options). Recommendation select (5 options). Sort select (6 options). Min Score slider (0-100, step 5). Reset button.
StockGrid	Responsive grid: 1 col (mobile), 2 cols (md), 3 cols (xl). SkeletonCard loading state (6 pulse cards). Empty state with SearchX icon. AnimatePresence for card transitions.
StockCard	Circular SVG score indicator (72px, color-coded: emerald >=80, amber >=50, red <50). 6 MiniScoreBars in 2-col grid. Price with gain from IPO %. Status + Recommendation badges. Hover: lift (-4px), gradient overlay, View Details button fade-in.
ScoreIndicator	Circular SVG progress ring. Configurable size. Color-coded progress stroke. MiniScoreBar: horizontal bar with label, color-coded fill.
StockDetail	Full-screen dialog. Info grid (8 cards). Total Score + 6 MiniScoreBars. Price Chart (Daily/Weekly toggle). Checklist (6 items, pass/fail). Risk Assessment (4 levels). Key Levels (Pivot, Support, IPO Price, Breakout Date).
PriceChart	Recharts ComposedChart: Area (emerald gradient, price) + Bar (slate gradient, volume at 1/4 height). Reference lines: Pivot (amber dashed), Support (red dashed), IPO Price (slate dashed). Custom tooltip with INR formatting. Volume format: K/L/Cr.
Checklist	6 items with thresholds: H-Pivot >=60, Vol Dry-Up >=50, Breakout Vol >=60, VCP >=50, vs IPO Low >=50, Weekly Conv >=50. Pass: green CheckCircle2. Fail: red XCircle. Pass count display.

StockListManager	Dialog for add/remove stocks. Two add modes: Yahoo Finance Search (with fallback validation) and Manual Entry (6 fields). Delete with confirmation. Current watchlist with symbol, rec badge, status, price, score. Info note about refreshing after adding.
ScannerProvider	React Query QueryClientProvider. staleTime: 30s, retry: 1. Wraps entire app.

11. Configuration Files

next.config.ts

```
output: "standalone" (for containerized deployment), typescript.ignoreBuildErrors: true,
reactStrictMode: false
```

tailwind.config.ts

```
darkMode: "class", content paths for app/components, CSS variable-based theme colors (hsl), chart
colors, tailwindcss-animate plugin
```

components.json (shadcn/ui)

```
style: "new-york", rsc: true, tsx: true, tailwind.cssVariables: true, baseColor: "neutral",
aliases: @/components, @/lib, @/hooks, iconLibrary: "lucide"
```

tsconfig.json

```
target: ES2017, module: esnext, moduleResolution: bundler, paths: @/* -> ./src/*, strict: true,
noImplicitAny: false
```

.env

```
DATABASE_URL=file:/home/z/my-project/db/custom.db
```

package.json scripts

Script	Command	Purpose
dev	next dev -p 3000 2>&1 tee dev.log	Development server with logging
build	next build && cp static + public + db to standalone	Production build with standalone output
start	node fork of .next/standalone/server.js (detached)	Production server, detached process
db:push	prisma db push	Push schema to database
db:generate	prisma generate	Generate Prisma client

12. Yahoo Finance Integration

The market data route (src/app/api/market-data/route.ts) integrates with Yahoo Finance v8 Chart API for real-time and historical data. The integration uses two Yahoo API endpoints: the chart API for price/OHLCV data and the search API for finding stocks. All requests include a User-Agent header to avoid blocking and use AbortSignal.timeout for timeout protection (8s for search/price, 15s for historical).

Symbol Mapping

Indian stocks on Yahoo Finance use a suffix format: .NS for NSE and .BO for BSE. The application maintains a symbol mapping dictionary in the toYahooSymbol() function for stocks whose Yahoo symbol differs from the internal symbol. Key mappings include: MTAR -> MTARTECH.NS, ADANIWILMAR -> AWL.NS, ETHOS -> ETHOSLTD.NS, PHARMEASY -> PHARMASY.NS. Unmapped symbols default to {SYMBOL}.NS for NSE or {SYMBOL}.BO for BSE.

Search Fallback

When Yahoo Finance search API returns no results for a query, the system falls back to direct symbol validation. It takes the uppercase query, appends .NS then .BO suffix, and tries to fetch a price via the chart API. If a valid price is returned, it creates a synthetic search result with the validated symbol. This handles cases where Yahoo search does not index certain Indian stocks but the chart API still serves their data.

Rate Limiting

To avoid Yahoo Finance rate limiting, the bulk refresh endpoints insert delays between stock fetches: 200ms for price-only refresh and 300ms for full historical refresh. The full refresh also has a fallback strategy: if historical data fetch fails, it tries the latest-price-only API as a fallback, ensuring partial data is still saved rather than failing completely.

13. Deployment Configuration

The application is configured for standalone deployment using Next.js output: "standalone" mode, which produces a minimal, self-contained server bundle in .next/standalone/. The build script also copies static assets, public files, and the SQLite database to the standalone directory. The production start script uses Node.js fork() with detached: true and unref() to keep the server process alive independently of the parent shell.

Caddy Reverse Proxy

A Caddyfile configures a reverse proxy on port 81 that forwards to localhost:3000. It also handles a special case for the XTransformPort query parameter, which allows routing to different ports for WebSocket connections. The proxy adds proper headers: Host, X-Forwarded-For, X-Forwarded-Proto, and X-Real-IP.

Database Considerations

The SQLite database file is located at db/custom.db and referenced via DATABASE_URL in .env. In serverless environments (like Vercel), this file is ephemeral and will be reset on each deployment. For persistent storage in such environments, a migration to PostgreSQL or another persistent database would be required. The current setup works well for single-server deployments (VPS, Docker container, or the current Z.ai platform). The Prisma client singleton pattern (storing on globalThis) prevents duplicate client creation during hot-reload in development.

14. Known Issues & Planned Features

Known Issues

- SQLite is ephemeral on serverless platforms (Vercel) - stock customizations are lost on redeployment. A localStorage-based persistence system was planned but the files (watchlist-persistence.ts, /api/sync/route.ts) are not currently present in the codebase. These need to be re-implemented.
- Yahoo Finance API can be unreliable - some stocks (ZOMATO, PHARMEASY in particular) may return no data. The fallback validation helps but does not always work.
- Prisma query logging is enabled in all environments (log: ["query"]) which adds overhead in production. Should be conditionally enabled only in development.

- The ScoreIndicator component uses absolute positioning without a relative parent, which can cause the score value to render outside its container in certain layouts.
- No authentication - anyone with access to the URL can add/remove stocks and modify the database.

Planned Features

- localStorage-based watchlist persistence: Store added/removed stocks in browser localStorage and sync with server on page load via a /api/sync endpoint. This would ensure customizations survive server restarts.
- PostgreSQL migration for production deployments on serverless platforms.
- Email/alert notifications when stocks hit breakout criteria.
- Historical score tracking to show how pillar scores evolve over time.
- Export functionality: CSV/Excel download of scan results.
- Comparison view: Side-by-side comparison of multiple stocks.
- Custom scoring weights: Allow users to adjust the weight of each pillar in the total score calculation.
- Automated scheduled scans with configurable frequency.